Day 14: Quorum Replication, Tunable Consistency & Read Repair

Balancing consistency and availability with quorum reads/writes,

Quorum Fundamentals

- N is the number of replicas that store each key.
- W is the minimum replicas required to accept a write.
- R is the minimum replicas that required to serve a read.

If R + W > N, read/write quorums overlap, Higher chance of reading the latest value.

Tunable Consistency

- Clients can choose "WEAK" or "STRONG" at request time.
- Stronger consistency uses larger R/W, but lowers availability & increases latency.
- Weaker consistency improves availability/ latency, but can return stale data.

Versioned Values

- Replicas can temporarily diverge due to failures or lag.
- A read collects multiple versions and picks the newest version.
- Version metadata (logical version, timeepstamp, vector clock) required.

Read Repair

- After a read finds the newest version, stale replicas, can be repaired in the same path.
- Read repair improves eventual convergence without a full anti-entrpy cycle.
- Repair-on-read is best for hot keys; cold keys still need background repair.

Sample Quorum Key-Value Write & Read

Put user:123~> A

1 Client makes write

k > W = 2 replicas accept

Write success

Replica 1

Replica 1 – V2

Latency

#8a5f207f - Av2 > - Av3

1 Client makes write

k > W = 2 replicas accept

3 Write success

Write success

2 Get user:123?

1 Client makes read

k > R = 2 replicas return latest version

A v2

Replica 2

Replica 2 Av2

Repair

#8a5f207f - Av2 > Av1

Repaired

1 Client makes read

k > R = 2 replicas return latest version

3 Versions 4v1 & 4v2 are repaired

Quorum and tunable consistency balance correctness against performance by controling read & write overlap.

System Design - Day 14

Quorum Replication, Tunable Consistency, and Read Repair

Prepared from Day-14 notes and sample implementation

Overview from README.md

Day 14: Quorum Replication, Tunable Consistency, and Read Repair

Day 14 focuses on how replicated databases balance consistency and availability using quorum reads/writes.

It covers:
• replication factor and quorum math (`N`, `R`, `W`)
• tunable consistency levels
• stale reads and version selection
• read repair for eventual convergence
• availability trade-offs under replica failures
• practical interview reasoning for Dynamo-style systems

In short, Day 14 is about serving reads and writes from replicated nodes while controlling correctness with quorum choices.

Core ideas

Quorum fundamentals
• `N` is the number of replicas that store each key.
• `W` is the minimum replicas required to accept a write.
• `R` is the minimum replicas required to serve a read.
• If `R + W > N`, read/write quorums overlap, which improves the chance of reading the latest value.

Tunable consistency
• Systems often let clients choose consistency levels per request.
• Stronger consistency uses larger `R`/`W`, but lowers availability and increases latency.
• Weaker consistency improves availability and latency, but can return stale data.

Versioned values
• Replicas can temporarily diverge due to failures or lag.
• A read collects multiple versions and picks the newest version.
• Version metadata (logical version/timestamp/vector clock) is required to resolve conflicts.

Read repair
• After a read finds the newest version, stale replicas can be repaired in the same path.
• Read repair improves eventual convergence without a full anti-entropy cycle.
• Repair-on-read is best for hot keys; cold keys still need background repair.

Day-14 sample: Quorum Key-Value Store

This repository includes a small quorum-based key-value sample in both Python and Java.

Functional requirements
• Write a key/value with configurable write quorum (`W`)
• Read a key/value with configurable read quorum (`R`)
• Track per-key versions to choose the latest value
• Simulate replica availability changes
• Perform read repair on stale replicas

High-level components

- `Replica`: in-memory key-value copy with availability state
- `Versioned value`: payload plus monotonic version
- `Quorum store`: coordinates reads/writes across replicas
- `Repair logic`: updates stale replicas after a read

Data flow

1. A write request increments the key version and replicates to available nodes
2. If available nodes are below `W`, write is rejected
3. A read gathers responses from available nodes
4. If available nodes are below `R`, read is rejected
5. The newest version is returned and stale responders are repaired

Project structure

```text
Day-14/
README.md
quorum-kv-sample/
python/
quorum_kv.py
demo.py
java/
QuorumKVStore.java
Main.java
```

Run the sample

Python

```powershell
cd quorum-kv-sample\python
python demo.py
```

Java

```powershell
cd quorum-kv-sample\java
javac Main.java QuorumKVStore.java
java Main
```

Interview takeaways

- Quorum settings are a business decision across consistency, latency, and availability
- `R + W > N` is a useful rule, not a complete guarantee without sane versioning/clock strategy
- Replica failures create divergence; read repair helps but does not replace background anti-entropy
- Consistency models should be explicit in API contracts, not implicit implementation detail
- Dynamo-style designs prioritize graceful degradation and eventual convergence under failure

Next improvements

- Add sloppy quorum and hinted handoff simulation
- Add vector clocks for concurrent write conflict resolution
- Add background anti-entropy loop independent of reads
- Add per-request consistency overrides (`ONE`, `QUORUM`, `ALL`)
- Add latency modeling to show user-visible trade-offs for each consistency level

Reference Implementation

Python and Java sample source files

File: quorum-kv-sample/python/demo.py

```python
from quorum_kv import QuorumKVStore


def print_states(store: QuorumKVStore, key: str) -> None:
    for line in store.states_for_key(key):
        print("  " + line)


def main() -> None:
    key = "user:42"
    store = QuorumKVStore(
        replica_names=["replica-a", "replica-b", "replica-c"],
        read_quorum=2,
        write_quorum=2,
    )

    print("initial write with all replicas available")
    print(f"  write success: {store.write(key, 'active')}")
    print_states(store, key)

    print("\nbring replica-c down and write a newer value")
    store.set_replica_availability("replica-c", False)
    print(f"  write success: {store.write(key, 'suspended')}")
    print_states(store, key)

    print("\nbring replica-c back and read the key")
    store.set_replica_availability("replica-c", True)
    value, version, repaired = store.read(key)
    print(f"  read value={value}, version={version}, repaired_replicas={repaired}")
    print_states(store, key)

    print("\nforce quorum failure and attempt write")
    store.set_replica_availability("replica-b", False)
    store.set_replica_availability("replica-c", False)
    print(f"  write success: {store.write(key, 'closed')}")
    print_states(store, key)


if __name__ == "__main__":
    main()
```

File: quorum-kv-sample/python/quorum_kv.py

```python
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True)
class VersionedValue:
    version: int
    value: str


class Replica:
    def __init__(self, name: str) -> None:
        self.name = name
        self.available = True
        self._store: dict[str, VersionedValue] = {}

    def set_available(self, available: bool) -> None:
        self.available = available

    def read(self, key: str) -> VersionedValue | None:
        return self._store.get(key)

    def apply_write(self, key: str, version: int, value: str) -> None:
        self._store[key] = VersionedValue(version=version, value=value)


class QuorumKVStore:
    def __init__(self, replica_names: list[str], read_quorum: int, write_quorum: int) -> None:
        if not replica_names:
            raise ValueError("replica_names must be non-empty")
        if read_quorum <= 0 or write_quorum <= 0:
            raise ValueError("read_quorum and write_quorum must be positive")
        if read_quorum > len(replica_names) or write_quorum > len(replica_names):
            raise ValueError("quorum values must be <= number of replicas")

        self._replicas = [Replica(name) for name in replica_names]
        self._read_quorum = read_quorum
        self._write_quorum = write_quorum
        self._next_version_by_key: dict[str, int] = {}

    def set_replica_availability(self, replica_name: str, available: bool) -> None:
        replica = self._find_replica(replica_name)
        replica.set_available(available)

    def write(self, key: str, value: str) -> bool:
        self._validate_key_value(key, value)
        available_replicas = self._available_replicas()
        if len(available_replicas) < self._write_quorum:
            return False

        version = self._next_version_by_key.get(key, 0) + 1
        self._next_version_by_key[key] = version
        for replica in available_replicas:
            replica.apply_write(key, version, value)
        return True

    def read(self, key: str) -> tuple[str | None, int | None, int]:
        if not key:
            raise ValueError("key must be non-empty")

        available_replicas = self._available_replicas()
        if len(available_replicas) < self._read_quorum:
            raise RuntimeError("read quorum not met")

        latest: VersionedValue | None = None
        for replica in available_replicas:
            current = replica.read(key)
            if current is None:
                continue
            if latest is None or current.version > latest.version:
                latest = current
```

```
    if latest is None:
        return None, None, 0

    repaired = 0
    for replica in available_replicas:
        current = replica.read(key)
        if current is None or current.version < latest.version:
            replica.apply_write(key, latest.version, latest.value)
            repaired += 1

    return latest.value, latest.version, repaired

def states_for_key(self, key: str) -> list[str]:
    lines: list[str] = []
    for replica in self._replicas:
        current = replica.read(key)
        if current is None:
            value_repr = "None"
            version_repr = "-"
        else:
            value_repr = current.value
            version_repr = str(current.version)

        lines.append(
            f"{replica.name}: available={replica.available}, version={version_repr}, value={value_repr}"
        )
    return lines

def _available_replicas(self) -> list[Replica]:
    return [replica for replica in self._replicas if replica.available]

def _find_replica(self, replica_name: str) -> Replica:
    for replica in self._replicas:
        if replica.name == replica_name:
            return replica
    raise ValueError(f"unknown replica: {replica_name}")

@staticmethod
def _validate_key_value(key: str, value: str) -> None:
    if not key:
        raise ValueError("key must be non-empty")
    if not value:
        raise ValueError("value must be non-empty")
```

File: quorum-kv-sample/java/Main.java

```java
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        String key = "user:42";
        QuorumKVStore store = new QuorumKVStore(
            Arrays.asList("replica-a", "replica-b", "replica-c"),
            2,
            2
        );

        System.out.println("initial write with all replicas available");
        System.out.println("  write success: " + store.write(key, "active"));
        printStates(store, key);

        System.out.println("\nbring replica-c down and write a newer value");
        store.setReplicaAvailability("replica-c", false);
        System.out.println("  write success: " + store.write(key, "suspended"));
        printStates(store, key);

        System.out.println("\nbring replica-c back and read the key");
        store.setReplicaAvailability("replica-c", true);
        QuorumKVStore.ReadResult readResult = store.read(key);
        System.out.println(
            "  read value="
                + readResult.getValue()
                + ", version="
                + readResult.getVersion()
                + ", repaired_replicas="
                + readResult.getRepairedReplicas()
        );
        printStates(store, key);

        System.out.println("\nforce quorum failure and attempt write");
        store.setReplicaAvailability("replica-b", false);
        store.setReplicaAvailability("replica-c", false);
        System.out.println("  write success: " + store.write(key, "closed"));
        printStates(store, key);
    }

    private static void printStates(QuorumKVStore store, String key) {
        for (String line : store.statesForKey(key)) {
            System.out.println("  " + line);
        }
    }
}
```

File: quorum-kv-sample/java/QuorumKVStore.java

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class QuorumKVStore {
    public static final class ReadResult {
        private final String value;
        private final Integer version;
        private final int repairedReplicas;

        private ReadResult(String value, Integer version, int repairedReplicas) {
            this.value = value;
            this.version = version;
            this.repairedReplicas = repairedReplicas;
        }

        public String getValue() {
            return value;
        }

        public Integer getVersion() {
            return version;
        }

        public int getRepairedReplicas() {
            return repairedReplicas;
        }
    }

    private static final class VersionedValue {
        private final int version;
        private final String value;

        private VersionedValue(int version, String value) {
            this.version = version;
            this.value = value;
        }
    }

    private static final class Replica {
        private final String name;
        private boolean available;
        private final Map<String, VersionedValue> store;

        private Replica(String name) {
            this.name = name;
            this.available = true;
            this.store = new HashMap<>();
        }

        private void setAvailable(boolean available) {
            this.available = available;
        }

        private VersionedValue read(String key) {
            return store.get(key);
        }

        private void applyWrite(String key, int version, String value) {
            store.put(key, new VersionedValue(version, value));
        }
    }

    private final List<Replica> replicas;
    private final int readQuorum;
    private final int writeQuorum;
    private final Map<String, Integer> nextVersionByKey;

    public QuorumKVStore(List<String> replicaNames, int readQuorum, int writeQuorum) {
        if (replicaNames == null || replicaNames.isEmpty()) {
            throw new IllegalArgumentException("replicaNames must be non-empty");
```

```
    }
    if (readQuorum <= 0 || writeQuorum <= 0) {
        throw new IllegalArgumentException("readQuorum and writeQuorum must be positive");
    }
    if (readQuorum > replicaNames.size() || writeQuorum > replicaNames.size()) {
        throw new IllegalArgumentException("quorum values must be <= number of replicas");
    }

    this.replicas = new ArrayList<>();
    for (String replicaName : replicaNames) {
        replicas.add(new Replica(replicaName));
    }
    this.readQuorum = readQuorum;
    this.writeQuorum = writeQuorum;
    this.nextVersionByKey = new HashMap<>();
}

public void setReplicaAvailability(String replicaName, boolean available) {
    Replica replica = findReplica(replicaName);
    replica.setAvailable(available);
}

public boolean write(String key, String value) {
    validateKeyValue(key, value);
    List<Replica> availableReplicas = availableReplicas();
    if (availableReplicas.size() < writeQuorum) {
        return false;
    }

    int version = nextVersionByKey.getOrDefault(key, 0) + 1;
    nextVersionByKey.put(key, version);
    for (Replica replica : availableReplicas) {
        replica.applyWrite(key, version, value);
    }
    return true;
}

public ReadResult read(String key) {
    if (key == null || key.isBlank()) {
        throw new IllegalArgumentException("key must be non-empty");
    }

    List<Replica> availableReplicas = availableReplicas();
    if (availableReplicas.size() < readQuorum) {
        throw new IllegalStateException("read quorum not met");
    }

    VersionedValue latest = null;
    for (Replica replica : availableReplicas) {
        VersionedValue current = replica.read(key);
        if (current == null) {
            continue;
        }
        if (latest == null || current.version > latest.version) {
            latest = current;
        }
    }

    if (latest == null) {
        return new ReadResult(null, null, 0);
    }

    int repaired = 0;
    for (Replica replica : availableReplicas) {
        VersionedValue current = replica.read(key);
        if (current == null || current.version < latest.version) {
            replica.applyWrite(key, latest.version, latest.value);
            repaired += 1;
        }
    }

    return new ReadResult(latest.value, latest.version, repaired);
}
```

```java
    public List<String> statesForKey(String key) {
        List<String> lines = new ArrayList<>();
        for (Replica replica : replicas) {
            VersionedValue current = replica.read(key);
            String valueRepresentation = current == null ? "None" : current.value;
            String versionRepresentation = current == null ? "-" : Integer.toString(current.version);

            lines.add(
                replica.name
                    + ": available="
                    + replica.available
                    + ", version="
                    + versionRepresentation
                    + ", value="
                    + valueRepresentation
            );
        }
        return lines;
    }

    private List<Replica> availableReplicas() {
        List<Replica> available = new ArrayList<>();
        for (Replica replica : replicas) {
            if (replica.available) {
                available.add(replica);
            }
        }
        return available;
    }

    private Replica findReplica(String replicaName) {
        for (Replica replica : replicas) {
            if (replica.name.equals(replicaName)) {
                return replica;
            }
        }
        throw new IllegalArgumentException("unknown replica: " + replicaName);
    }

    private static void validateKeyValue(String key, String value) {
        if (key == null || key.isBlank()) {
            throw new IllegalArgumentException("key must be non-empty");
        }
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException("value must be non-empty");
        }
    }
}
```